

MSA106

Giada Arosio – A00168823

Assessment 2

DevOps: Ongoing Running and Maintenance

The DevOps strategy for AstroLearn focuses on maintaining the system's reliability while supporting future scalability. The goal is to keep the application lightweight and to introduce automation and operational improvements gradually as it grows. Each improvement aims to make the architecture easier to monitor, update, and scale without interrupting the service.

Telemetry is already integrated into the architecture to observe the system's ongoing performance. Each microservice returns its status (health) and logs basic metrics such as latency, error rates, and uptime. These checks are collected to help identify faults before they affect users. Newman (2015) highlights that monitoring and automation are essential parts of maintaining operational stability in microservice systems. In AstroLearn, this approach provides visibility over all running components and forms the foundation for further DevOps practices.

At this stage, testing is manual and supported by telemetry. As AstroLearn evolves, automated testing will be introduced through a CI/CD pipeline. This pipeline automatically builds, tests, and deploys new updates, ensuring that changes are safe and repeatable. Newman (2015) explains that continuous integration and delivery help reduce deployment risk by verifying every change before release.

Automated performance testing will also be considered. Salvadori et al. (2016) describe an architecture where each microservice is tested independently to detect performance bottlenecks early. Adopting a similar approach will maintain stability as AstroLearn's traffic and service count increase.

Although the current architecture does not require load balancing, the API Gateway can be horizontally scaled in future, when demand increases, by deploying multiple instances behind a load balancer to distribute incoming traffic and preventing single points of failure. Newman (2015) notes that scaling stateless services horizontally improves both resilience and responsiveness. This change will make AstroLearn capable of handling higher loads without impacting performance.

Asynchronous communication now relies on simple REST calls. In future versions, a message broker such as RabbitMQ or Kafka could manage event queues, providing more reliable data exchange between services and improving high traffic handling. According to Newman (2015), using brokers helps to reduce the risk of failures and simplifying maintenance.

Currently, authentication is managed internally. Over time, it can be delegated to an external identity provider to improve security and simplify user management. For example, users could log in through trusted external accounts such as Google or Microsoft, reducing the

need for AstroLearn to manage passwords directly. This change would reduce maintenance effort and ensure up-to-date authentication standards.

Moreover, notifications are retrieved manually within the app. Future updates could implement real-time push notifications, using WebSocket or Firebase, to provide instant alerts. This enhancement will make communication and user experience more dynamic.

All services currently share one physical database but store their data in separate logical spaces. As the system grows, the shared database could be split into separate physical databases for each service without changing the architecture, ensuring full scalability and data independence. Newman (2015) notes that independent storage per service aligns with deployable autonomy and simplifies ongoing maintenance.

References

Newman, S. (2015). *Building microservices: designing fine-grained systems*. O'Reilly Media.

<https://book.northwind.ir/bookfiles/building-microservices/Building.Microservices.pdf>

Salvadori, I., Mello, R., De Camargo, A., & Siqueira, F. (2016). *An architecture to automate performance tests on microservices*. ACM. <https://doi.org/10.1145/3011141.3011179>